Equipe e Divisão

RM: 573875 - João Lucas Silva Lopes – tarefa: Programação

RM: 571794 - Alan Otalvaro – tarefa: Atualização do prompts

RM: 570562 – João Pedro Ribeiro – tarefa: criação do documento rag

RM: 569429 – Enzo Ribeiro – tarefa: Responsável por outra sprint

RM: 573899 – João Pedro Evangelista – tarefa: Responsável por outra sprint

RM: 573195 - Gabriel de Paula Santos – tarefa: Responsável por outra sprint

Evolução

Nas verões anteriores do projeto, tínhamos um aplicativo de interface de terminal que seguia a estrutura de um chat de bate papo, a tarefa da ia era ajudar o usuário com qualquer pergunta generalista, podendo fornecer recomendações de postos para uso, além disso, todas as respostas eram avaliadas com base na acurácia, que era calculada a partir da quantidade de palavras chaves predeterminadas presentes na resposta, tendo uma classificação variante de adequado ou não a partir desse valor, o problema presente nesse teste era que para avaliar a resposta da ia era necessário já ter terminado a pergunta e a resposta mais adequada para ela.

Agora, na implementação para a sprint 4, refaturamos a ia para seguir estritamente um rag, ignorando o aspecto de recomendação de postos e generalista anterior, agora a ia foca estritamente no suporte ao cliente, tendo casos e respostas previamente documentados para serem seguidos, no que se trata de interface, a interface de terminal original foi substituída pelo streamlit, mas ainda se mantêm utilizado na execução de testes, esses mesmos testes que foram aprimorados através do ragas, acurácia foi deixado de lado, priorizando faithfulness e answer_relevancy, o que permite testes em qualquer tipo de pergunta relacionada ao documento de suporte técnico, bastando associar a mensagem a seção contendo a resposta, eliminando a limitação anterior.

Pipeline rag

Decisões

Chunking	Overlap
1024	100

Para montar o rag no projeto, optamos pelos valores de 1024 para chunk e 100 para overlap, ambos foram escolhidos para serem valores consideravelmente grandes para conseguirem conter cada seção do documento corretamente

Base de conhecimento

Para a base de conhecimentos, escolhemos criar um documento focado em suporte técnico, contendo questões voltados a duvidas e problemas que um usuário poderia ter, seus principais indicadores e as ações ou respostas recomendas a serem dados pelo usuário

Trade offs

Para implementar o rags no projeto tivemos que tomar algumas ações que mudaram a estrutura original, a principal foi o abandono do sistema de recomendação, já que as resposta da ia seriam voltados ao documento técnico, a injeção de dados sobre os postos e a ia intermediaria de classificação criavam um custo extra para o projeto, já que antes da resposta final ser gerada, ainda era necessário fazer uma outra requisição para a api, o que gastava processamento e tokens, com o corte nessa função, o pipeline agora só tem duas partes, a classificação da pergunta e a geração da resposta, ficando com suas ias para essa função e dois prompts destintos.

Problemas e soluções

Embeddings

Devido a certas limitações na nossa conta do ollama cloud, tivemos que implementar o modelo de embeddings localmente.

LLM do ragas

Originalmente planejávamos usar apenas a api do gemini no projeto para evitar dependências extras, mas os principais modelos gratuitos carecia de certas dependências que o ragas exigia, sendo então preciso usar o ollama cloud para executar os testes, com o gemini gerando as respostas para os testes e o gemma4 vindo do ollama sendo usado para avaliar essas respostas pelo ragas